Updating and Optimizing Mutation Detection Pipeline

Snakemake 7 → 9 on SLURM

Alan Chapman — HPC Systems Analyst
Arizona State University Research Computing

RMACC HPC Symposium 2026 · Boise, ID

DETECT in 30 Seconds

DNM Extraction Through Empirical Cutoff Thresholds

- Pfeifer Lab, Arizona State University
- Identifies *de novo* mutations (new in a child, absent in either parent) in **simulated** trio sequencing data
- Workflow: simulate trio reads → inject known DNMs → align (BWA) → variant call (GATK) → recommend filter thresholds
- Output is a set of **summary statistics / empirical thresholds** that downstream users then apply to **real** trio sequencing data

Why HPC matters: a single production run is hundreds of jobs and ~20–26 hours wall time on SLURM. Reliability at that scale is the whole game.

Talk scope: the *migration mechanics* — how the SLURM integration changed between Snakemake 7 and 9, with the actual code.

Why Bother — Results First

Dataset	Snakemake 7	Snakemake 9	Δ
Demo	31m 52s	**24m 29s**	23% faster
Production	26h 4m	**19h 41m** (avg/3)	24% faster

Stability — three production runs, Sol cluster, Feb 2026

Run	Jobs	Completed	Failed
5DETECT	183	183	0
6DETECT	362	362	0
7DETECT	362	362	0
Total	**907**	**907**	**0**

The 24% isn't faster code. It's better scheduling, retry recovery, and right-sized resources. The rest of the talk is *how*.

What `--cluster` Actually Was

In Snakemake 7, `--cluster` took a **shell string** that Snakemake would interpolate per job and hand to your scheduler.

```
snakemake --cluster "sbatch -n {threads} --mem={resources.mem_mb} \
                            -t 01:00:00 \
                            -o logs/{rulename}.{jobid}.out \
                            -e logs/{rulename}.{jobid}.err"
```

Mental model: Snakemake renders that string, calls `sbatch`, parses the returned job ID, then **polls** `squeue` / `sacct` to track state. No real SLURM awareness — just shell-out and string-match.

That worked. But it was the source of every operational pain that follows.

The Real DETECT Snakemake 7 Invocation

Why It Hurt at Scale

- **No rate limiting** — Snakemake fires `sbatch` and `squeue` as fast as the DAG allows. SLURM's slurmctld socket → timeouts and `slurm_load_jobs error: Socket timed out` failures. Random rules die.
- **Hardcoded walltime** — `-t 01:00:00` for *every* rule. The 4-hour assembly-depth job needed its own one-off submission.
- **String-typed resources** in the Snakefile — `runtime='15m'`, `runtime='4d'`, `runtime='8h'`. No retry backoff.
- **No partition intelligence** — every job goes to the default queue, regardless of fit.
- **Bare `python` in shell rules** — picks up whatever's first on `$PATH` inside the SLURM job. Worked on the login node, broke on compute nodes.
- **Failure diagnosis = `grep` archaeology** across 800 log files.

The Breaking Change

- **Snakemake 8 (Feb 2024):** `--cluster` deprecated. Executor plugin interface introduced.
- **Snakemake 9 (Sep 2024):** `--cluster` **removed**.
- Every HPC-bound Snakemake 7 workflow needs migration. There is no `--cluster` shim.

The new model: scheduler integration moves out of the CLI flag and into a **plugin** that Snakemake loads.

```
pip install snakemake-executor-plugin-slurm
snakemake --executor slurm ...
```

The plugin speaks SLURM natively — submits via `sbatch`, tracks state via `sacct`, handles retries, applies rate limits,

What the Plugin Gives You for Free

- **Native job submission** — no template string. Resources go through the plugin's typed interface.
- **Native status polling** — proper `sacct` queries with backoff, not hammering `squeue` in a loop.
- **Rate limiting** as a first-class config knob.
- **Automatic partition selection** when given a partitions YAML.
- **Retry semantics** wired into Snakemake's own attempt counter.
- **Profile-based config** — submission settings live in YAML, not in your shell history.

There's also `snakemake-executor-plugin-cluster-generic` (a `--cluster` workalike). **Don't use it for new work.** It exists for the migration off-ramp; the SLURM plugin is the destination.

Invocation: Before and After

Snakemake 7

```
sbatch --wrap "snakemake --configfile config.json \
  --cluster 'sbatch -n {threads} --mem={resources.mem_mb} -t 01:00:00 ...' \
  -j 100 --latency-wait 60 --rerun-incomplete"
```

Snakemake 9

```
snakemake -p --snakefile Snakefile \
    --configfile work/config/config.json \
    --executor slurm \
    --workflow-profile profiles/slurm \
    --slurm-partition-config profiles/slurm/public.yaml \
    --scheduler greedy -j 100 --cores 100 \
    --latency-wait 30 --retries 3 --rerun-incomplete
```

Two new flags carry most of the meaning: `--workflow-profile` and `--slurm-partition-config`. Everything else lives in YAML now.

`profiles/slurm/config.yaml` — Core

```yaml
executor: slurm
jobs: 150
cores: 150
retries: 1
latency-wait: 30
rerun-incomplete: true
scheduler: greedy
slurm-logdir: logs

# rate limiting — the cure for socket timeouts
max-jobs-per-timespan: 9/1s
max-status-checks-per-second: 2

# defaults for any rule that doesn't override them
default-resources:
  - mem_mb=8000
  - runtime=30
  - cpus_per_task=8
```

The whole `--cluster` template string is now declarative. `slurm-logdir` replaces every per-rule `-o` / `-e` argument you used to hand-write.

`profiles/slurm/config.yaml` — Per-Rule Resources

```yaml
set-resources:
  SplitFasta:
    mem_mb: 4000
    runtime: 10
    cpus_per_task: 1

  ReformatMutations:
    mem_mb: 8000
    runtime: 20
    cpus_per_task: 1

  AssemblyDepth:
    mem_mb: 64000
    runtime: 480
    cpus_per_task: 16
```

Right-sizing in YAML, not in the Snakefile. Means the workflow author and the cluster operator can tune resources independently — and rolling back a bad tuning is a `git revert` on one file.

Automatic Partition Selection

```yaml
# profiles/slurm/public.yaml
partitions:
  public:
    max_runtime: 10080        # 7 days
    max_mem_mb: 500000        # 500 GB
    max_cpus_per_task: 120
    max_nodes: 24
  htc:
    max_runtime: 240          # 4 hours
    max_mem_mb: 256000
    max_cpus_per_task: 120
    max_nodes: 24
```

Pass with `--slurm-partition-config public.yaml`. The plugin matches each job's resource request against the table and picks the smallest partition that fits — the 10-minute `SplitFasta` job lands on `htc`, the 8-hour `AssemblyDepth` job lands on `public`. **No more hardcoded `-p` flag; no more rule-by-rule partition logic.**

Resources: From Strings to Lambdas

Snakemake 7 — string-typed, fixed

```
rule MutateOffspring:
    resources:
        mem_mb = 8000,
        runtime = '15m'          # string format
```

Snakemake 9 — integer minutes, with `attempt`-based backoff

```
rule callHaplotypeCaller:
    resources:
        mem_mb  = lambda wildcards, attempt: 32000 + (16000 * (attempt - 1)),
        runtime = lambda wildcards, attempt: 60    + (30    * (attempt - 1))
```

- Attempt 1: 32 GB / 60 min → OOM kill, SLURM exits with retry-eligible code
- Attempt 2: 48 GB / 90 min → likely succeeds
- Attempt 3: 64 GB / 120 min → upper bound

Result on DETECT: four transient OOM/walltime failures across 907 jobs, all auto-recovered. Zero manual intervention.

Containers: Per-Rule → Global

Snakemake 7 — repeat the directive on every GATK rule

```
rule MarkDuplicates:
    singularity: "container/gatk4.6.2.sif"
    shell: "gatk MarkDuplicates ..."

rule BQSR:
    singularity: "container/gatk4.6.2.sif"
    shell: "gatk BaseRecalibrator ..."
```

Snakemake 9 — global directive at the top of the Snakefile

```
GATK_CONTAINER = "container/gatk4.6.2.sif"
container: GATK_CONTAINER

rule MarkDuplicates:
    shell: "gatk MarkDuplicates ..."
```

Drives consistency. Adding a new GATK rule is no longer a chance to forget the directive and have it silently use whatever GATK happens to be on `$PATH`.

The Python-Path Gotcha

Not a Snakemake-version issue — a discipline issue. The migration was the forcing function.

Symptom: `python scripts/mutator.py` works on the login node, fails inside a SLURM job with `ModuleNotFoundError`. The compute node has a different `$PATH`.

Before — bare `python` (what the Sn7 Snakefile actually shipped)

```
shell: "python {snakedir}/scripts/mutator.py -i {input} ..."
```

After — resolve from `$CONDA_PREFIX` at workflow load

```
def get_python_executable():
    conda_prefix = os.environ.get("CONDA_PREFIX", "")
    if conda_prefix:
        candidate = os.path.join(conda_prefix, "bin", "python3")
        if os.path.exists(candidate):
            return candidate
    return "python3"

python_exec = get_python_executable()

rule MutateOffspring:
    params: python_exec = python_exec
```

Rate Limiting in Practice

```
max-jobs-per-timespan: 9/1s
max-status-checks-per-second: 2
```

Two lines in `profiles/slurm/config.yaml`. They eliminated the failure mode that had been the dominant source of run failures.

Before: Snakemake fires hundreds of `sbatch` and `squeue` calls when a wide rule fans out. slurmctld saturates. Calls time out. Snakemake interprets the timeout as a failed submission. Random rules silently don't run.

After: the plugin enforces ≤9 submissions per second and ≤2 status checks per second cluster-wide. slurmctld never sees a burst. Run failures from this class: zero across 907 jobs.

If you take one thing from this talk, it's **set these two values.**

`run.sh` and `sbatch.sh`

Two thin wrappers. The deck doesn't have room for both — the shape is:

```
#!/bin/bash
set -euo pipefail
module load mamba/latest
source activate DETECT

snakemake -p --snakefile Snakefile \
    --configfile work/config/config.json \
    --executor slurm \
    --workflow-profile profiles/slurm \
    --slurm-partition-config profiles/slurm/public.yaml \
    --scheduler greedy -j 100 --cores 100 \
    --latency-wait 30 --retries 3 --rerun-incomplete \
    2>&1 | tee logs/output.log
```

- **`run.sh`** — runs on the login node. For testing.
- **`sbatch.sh`** — same body, wrapped for `sbatch`. The `snakemake` driver itself becomes a SLURM job.

Either way, the *contents* of the snakemake call are the same. Configuration lives in the profile.

Where the 24% Came From

Same code, same data, same cluster — different scheduler behavior.

- **Right-sized resources** — light rules like `SplitFasta` no longer request 64 GB and wait for a fat node. They land on `htc` in seconds.
- **Partition fit** — short jobs go to `htc`, long jobs go to `public`. Queue waits drop because requests match what's available.
- **Retry recovery** — transient OOMs add a few minutes, not a full re-run. Snakemake 7 had no automatic retry on resource failures.
- **No socket-timeout reruns** — every job we submit actually reaches SLURM the first time.

The Snakemake 9 executor plugin is doing what `--cluster` couldn't: behaving like a citizen of the cluster.

Migration Checklist

If you're sitting on a Snakemake 7 workflow today:

1. `pip install snakemake-executor-plugin-slurm`
2. Drop `--cluster '...'`. Add `--executor slurm --workflow-profile profiles/slurm`.
3. Create `profiles/slurm/config.yaml` — start with `executor: slurm`, `jobs:`, `default-resources:`.
4. **Add the two rate-limit lines.** Now, before anything else.
5. Move per-rule resources from Snakefile `resources:` into profile `set-resources:`.
6. Convert `runtime: '4h'` strings to integer minutes. Wrap in `lambda wildcards, attempt: ...` for backoff.
7. Hoist per-rule `singularity:` directives to a global `container:` where appropriate.
8. Replace bare `python` in shell rules with a `get_python_executable()` resolved at load time.
9. Add `--slurm-partition-config partitions.yaml` once you have more than one partition shape.
10. Run a single-chromosome dry-run before a full submission.

References

This work

- DETECT — Pfeifer Lab, Arizona State University

Snakemake

- Snakemake docs: `snakemake.readthedocs.io`
- SLURM executor plugin: `github.com/snakemake/snakemake-executor-plugin-slurm`
- Plugin interface spec: `github.com/snakemake/snakemake-interface-executor-plugins`

Migration discussion

- Snakemake 8 release notes (the `--cluster` deprecation announcement)
- `snakemake --help-all` after installing the SLURM plugin lists every `--slurm-*` flag

Thank You

Questions?

Alan Chapman — `alan.chapman@asu.edu`
HPC Systems Analyst, ASU Research Computing

DETECT — Pfeifer Lab, Arizona State University